

Domain Model

Version: 1.0

Purpose

This document defines the business domain model of Trading Platform Pro and its primary application, the Trading Cockpit.

The Domain Model represents trading concepts, business rules, state transitions and invariants.

The Domain remains independent from technical implementation details, UI frameworks, persistence technologies and external providers.

Domain Philosophy

The Domain contains business knowledge.

It defines:

- Entities
- Value Objects
- Aggregates
- Domain Services
- Domain Rules
- Domain Events
- Domain Exceptions

Technical concerns do not belong in the Domain.

The Domain shall use explicit trading language.

Core Domain Capabilities

The initial Trading Cockpit domain is organized around the following business capabilities:

- Instruments
- Watchlists
- Trading Candidates
- Trading Decisions
- Portfolio
- Positions
- Orders
- Risk
- Trading Review

Market Data supports trading workflows but external market data acquisition remains an infrastructure capability.

Workspace management, configuration and logging are not business domains.

Instruments

The Instrument capability represents tradeable financial instruments.

Instrument

An Instrument has identity and represents a financial product.

Typical attributes may include:

- InstrumentId
- Symbol
- Name
- InstrumentType
- Exchange

- Currency

Instrument identity shall remain independent from a specific broker identifier where practical. Broker-specific identifiers belong to integration mappings.

Instrument Invariants

Examples:

- InstrumentId shall be valid.
- Symbol shall not be empty.
- Currency shall be valid where required.
- Instrument identity shall remain stable.

Watchlists

The Watchlist capability represents user-defined collections of instruments.

Watchlist

A Watchlist is an Aggregate Root.

Typical state:

- WatchlistId
- Name
- Instruments
- CreatedAt
- UpdatedAt

Watchlist Rules

A Watchlist shall:

- have identity
- have a valid name
- contain valid Instrument references
- prevent unintended duplicate entries
- preserve explicit instrument ordering where supported

Watchlist Events

Examples:

- WatchlistCreated
- InstrumentAddedToWatchlist
- InstrumentRemovedFromWatchlist
- WatchlistRenamed

Trading Candidates

A Trading Candidate represents an instrument currently under trading evaluation. Trading Candidates bridge market observation and explicit trading decisions.

Trading Candidate

A Trading Candidate is an Aggregate Root.

Typical state:

- CandidateId
- InstrumentId
- CandidateStatus
- Origin

- Tags
 - CreatedAt
 - UpdatedAt
- Possible origins may include:
- Watchlist
 - Market Scanner
 - Manual Selection
 - Signal

Candidate Status

Initial candidate states may include:

- New
- Reviewing
- Accepted
- Rejected
- Archived

State transitions shall remain explicit.

Trading Candidate Rules

A Trading Candidate shall:

- reference a valid Instrument
- have a defined lifecycle state
- preserve its origin where available
- preserve creation and update timestamps

Trading Candidate Events

Examples:

- TradingCandidateCreated
- TradingCandidateReviewStarted
- TradingCandidateAccepted
- TradingCandidateRejected
- TradingCandidateArchived

Trading Decisions

A Trading Decision represents an explicit trading decision associated with a trading candidate or instrument.

The decision process shall remain transparent and reviewable.

Trading Decision

A Trading Decision is an Aggregate Root.

Typical state:

- DecisionId
- CandidateId
- InstrumentId
- DecisionStatus
- Notes
- Tags
- CreatedAt
- UpdatedAt

Possible decision states may include:

- Draft

- Accepted
- Rejected
- Cancelled
- Executed
- Reviewed

Trading Decision Rules

A Trading Decision shall:

- reference a valid Instrument
- have an explicit status
- preserve relevant decision state
- preserve decision timestamps

An accepted decision does not automatically imply successful order execution.

Decision state and order state shall remain separate.

Trading Decision Events

Examples:

- TradingDecisionCreated
- TradingDecisionAccepted
- TradingDecisionRejected
- TradingDecisionCancelled
- TradingDecisionExecuted
- TradingDecisionReviewed

Portfolio

The Portfolio capability represents the user's trading portfolio from a business perspective.

Portfolio

Portfolio is an Aggregate Root.

Typical state may include:

- PortfolioId
- AccountReference
- Positions
- Currency
- ValuationState
- UpdatedAt

The Domain shall distinguish between known and unavailable financial values.

Missing financial information shall not silently become zero.

Portfolio Rules

Examples:

- Portfolio identity shall remain stable.
- Position references shall remain consistent.
- Financial values shall preserve currency context.
- Unavailable values shall remain explicitly unavailable.

Portfolio Events

Examples:

- PortfolioUpdated
- PortfolioValuationUpdated

- PortfolioStateChanged

Positions

The Position capability represents an economic position in an instrument.

Position

Position is an Aggregate Root where independent lifecycle rules are required.

Typical state may include:

- PositionId
- InstrumentId
- Quantity
- AveragePrice
- PositionStatus
- OpenedAt
- ClosedAt

Possible states may include:

- Open
- Closing
- Closed

Additional states require explicit business justification.

Position Rules

Examples:

- Position shall reference a valid Instrument.
- Quantity semantics shall remain explicit.
- State transitions shall remain valid.
- Closed positions shall not silently return to Open.
- Missing average price shall remain unavailable.

Position Events

Examples:

- PositionOpened
- PositionUpdated
- PositionClosing
- PositionClosed

Orders

The Order capability represents the business lifecycle of a trading order.

Order

Order is an Aggregate Root.

Typical state may include:

- OrderId
- InstrumentId
- Action
- Quantity
- OrderType
- PriceParameters
- OrderStatus
- CreatedAt

- UpdatedAt

Possible order states may include:

- Draft
- Validated
- Submitted
- Acknowledged
- PartiallyFilled
- Filled
- CancelPending
- Cancelled
- Rejected

Order Invariants

Examples:

- Order shall reference a valid Instrument.
- Quantity shall satisfy order rules.
- Required price parameters shall exist for the selected order type.
- Invalid orders shall not enter Submitted state.
- Filled orders shall not return to Submitted.
- Rejected orders shall preserve rejection state.

Order Validation

Order validation is domain logic where validation depends on business rules.

Examples:

- valid quantity
- valid action
- valid order type
- required price parameters
- supported instrument constraints

Broker-specific technical validation remains in the broker adapter.

Order Events

Examples:

- OrderCreated
- OrderValidated
- OrderSubmitted
- OrderAcknowledged
- OrderPartiallyFilled
- OrderFilled
- OrderCancellationRequested
- OrderCancelled
- OrderRejected

Risk

The Risk capability represents available trading and portfolio risk context.

Risk information may include:

- Portfolio Exposure
- Position Exposure
- Concentration
- Margin Context
- Risk Limits

Risk values shall preserve:

- value
 - source context where business-relevant
 - availability state
 - timestamp where time-dependent
- Unavailable risk information shall remain explicitly unavailable.

Risk Rules

Examples:

- risk values shall preserve measurement context
- incompatible currencies shall not be combined silently
- unavailable values shall not be treated as zero
- risk limit violations shall remain explicit

Risk Events

Examples:

- RiskContextUpdated
- RiskLimitExceeded
- RiskStateChanged

Trading Review

Trading Review represents the business capability for reviewing trading decisions and outcomes. It connects historical:

- Trading Candidates
- Trading Decisions
- Orders
- Executions
- Positions
- Outcomes

Trade Review

A Trade Review may represent a completed or reviewable trading workflow.

Typical state may include:

- ReviewId
- DecisionId
- InstrumentId
- ReviewStatus
- Notes
- Tags
- OutcomeReference
- CreatedAt
- UpdatedAt

Trading Review Rules

A review shall preserve references to the historical trading workflow.

Historical business state shall not be silently rewritten to match later state.

Corrections shall remain traceable where practical.

Trading Review Events

Examples:

- TradeReviewCreated

- TradeReviewUpdated
- TradeReviewCompleted

Value Objects

Value Objects represent immutable business values.

Initial Value Objects may include:

- InstrumentId
- WatchlistId
- CandidateId
- DecisionId
- PortfolioId
- PositionId
- OrderId
- Symbol
- Price
- Quantity
- Money
- Currency
- Percentage
- Timestamp

Value Objects shall be immutable.

Value Objects shall validate their own invariants where appropriate.

Money

Money represents a monetary amount with currency context.

Money shall contain:

- Amount
- Currency

Money values with incompatible currencies shall not be combined silently.

Currency conversion requires explicit conversion context.

Price

Price represents an instrument price.

Price shall preserve numeric precision appropriate to the business requirement.

Price shall not contain provider-specific transport logic.

Quantity

Quantity represents an instrument quantity.

Quantity semantics may depend on instrument type.

The Domain shall avoid implicit assumptions that all quantities represent whole shares.

Percentage

Percentage represents a normalized percentage value.

The internal representation shall be defined consistently.

The system shall avoid mixing decimal and display percentage semantics implicitly.

Timestamp

Timestamp represents business-relevant time.

Timezone context shall remain explicit.

The Domain shall not silently mix timezone semantics.

Aggregates

Initial Aggregate Roots include:

- Watchlist
- Trading Candidate
- Trading Decision
- Portfolio
- Position
- Order

Trade Review may become an Aggregate Root when review lifecycle requirements justify independent consistency boundaries.

Aggregate boundaries shall be based on business consistency requirements.

Avoid large aggregates that coordinate unrelated capabilities.

Domain Services

Domain Services contain business logic that does not naturally belong to one Entity or Value Object.

Potential Domain Services include:

- Order Validation
- Risk Evaluation
- Portfolio Consistency Evaluation
- Position State Evaluation

Domain Services shall remain stateless where practical.

Technical orchestration belongs to the Application layer.

Domain Events

Domain Events represent meaningful business state changes.

Events shall:

- use business language
- be immutable
- represent completed state changes
- contain sufficient business identity
- avoid infrastructure-specific information

Examples:

- TradingCandidateCreated
- TradingDecisionAccepted
- OrderSubmitted
- OrderFilled
- PositionOpened
- PositionClosed
- RiskLimitExceeded

Domain Events shall not be used as arbitrary method-call replacements.

Repository Abstractions

Repository abstractions may be defined in the Domain or Application layer depending on ownership and use-case requirements.

Implementations belong to Infrastructure.

Example:

```
```text
```

```
OrderRepository
```



SqlAlchemyOrderRepository